

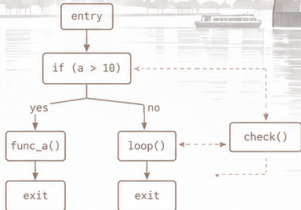
```
; disassembly
00401000 <_start>:
401000: 55          push    rbp
401001: 48 89 e5    mov     rbp, rsp
401004: 48 83 ec 20  sub     rsp, 0x20
401008: bf 01 00 00 00 mov     edi, 0x1
40100d: e8 1e 00 00 00 call    401030 <main>
401012: b8 3c 00 00 00 mov     eax, 0x3c
401017: 0f 05      syscall
```

```
; main
00401030 <main>:
401030: 55          push    rbp
401031: 48 89 e5    mov     rbp, rsp
...
```

Inside Linux GoGra backdoor

Axelle Apvrille, Fortinet

44con, September 2026



Who am I?



This image is AI-generated.

No way I look like that, right?

- Principal security researcher with **Fortinet**
- I reverse incoming **mobile malware** (Android, iOS) and **IoT malware**
- Founder of **Ph0wn CTF**

FORTINET®



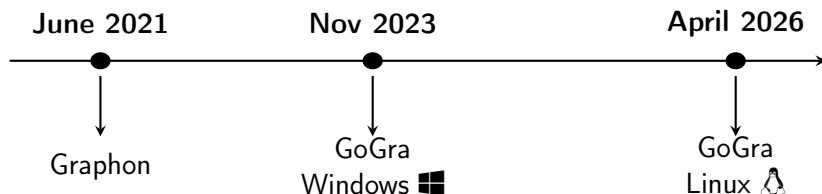
- Backdoor family,
- Targets mostly **South Asia, India and Afghanistan,**
- Implemented in **Go,**
- Abuses **Microsoft Graph API:** REST API to access Office 365 resources.



All images of **Pico le Croco**  are hand made (by myself)

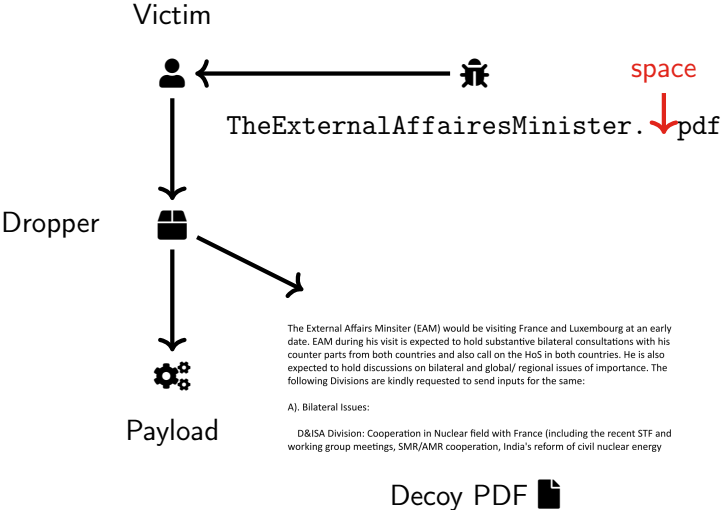


Chronology



- **Graphon** is an *ancestor*: .NET backdoor.
- Both attributed to **Harvestor** APT group
- *Harvester* is believed to be a nation-state-backed APT group. Targeting telecom, IT and government sectors.

GoGra Attack chain



Contribution - What are you going to learn?

Linux/GoGra is presented in a **technical blog** 

Why talk more about it?



The blog post explains the malware from a **global** perspective



Details are not covered
e.g. how does it really
achieve this or that?



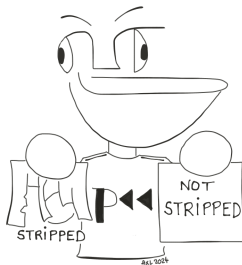
We explain **how we reverse**, so you can do it later

Inside Linux GoGra backdoor

- 1 Introduction: who am I, what is GoGra, contribution
- 2 **Reversing Go binaries**
- 3 Dropper's extraction mechanism
- 4 Executing the payload
- 5 Persistence
- 6 Communication through encrypted e-mails



Reversing Go is difficult



- Decompilers don't handle Go **strings** correctly.
- Go uses a **stack-based ABI** on i386. This confuses Ghidra.
- Binaries are **stripped**. Some information is recoverable in the `.gopclntab`, but Ghidra doesn't know how to do that.
- Reversing Go: [Asher Davila, 2026](#) 

AI Setup



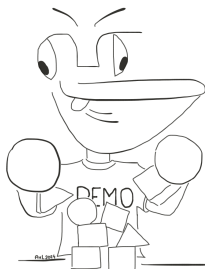
Ghidra MCP 



Select a LLM which is good with code e.g. Claude Sonnet 4.5+, Qwen 3.8

You can't reverse without a disassembler!

Demo Takeaway



- 1 **Everything begins with disassembly.** Not with AI.
- 2 We connect Ghidra to our AI harness.
- 3 Ask specific questions to AI.
- 4 **AI is irritating.**
- 5 In the end, it does the job.

Ghidra offers no function names

FUN_08049000
FUN_08049080
FUN_08049150
FUN_08049230
FUN_08049280
FUN_08049300
FUN_080493a0
FUN_080494a0
FUN_08049b50
FUN_08049b90
FUN_0804a050
FUN_0804a870
FUN_0804a890

```
file ./gogra.elf
./gogra.elf: ELF 64-bit LSB executable, x86-64 ..
↳ stripped
```

- Stripped ELF: symbols are gone.
- But the binary obviously includes some **function names**:

```
$ strings ./gogra.elf | grep main
main.main
main.filedropping
main.creatingDesktopfile
main.CreateServiceFile
main.DropEmbeddedFile
main.OpenFile
```

Function metadata

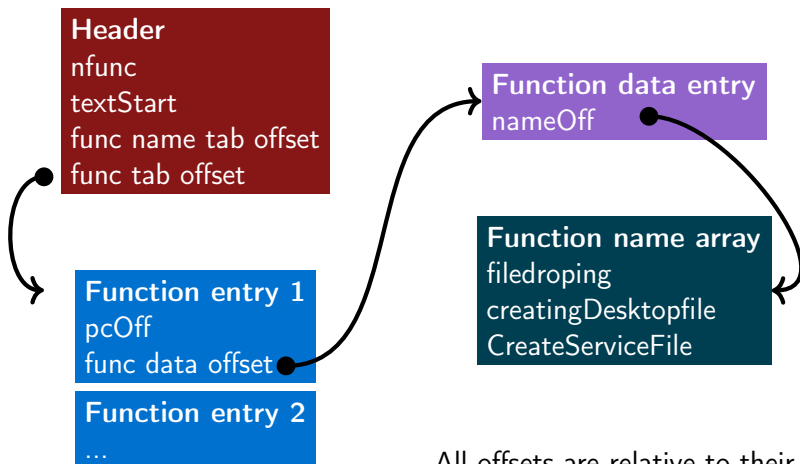
- Go stores **function metadata** in a special tab: pcIntab (since 2014)
- PcIntab **source code** 
- Ghidra doesn't know how to parse it, but **researchers do** 





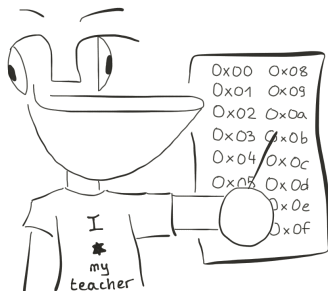
Hang on!
It gets *slightly* technical.
But it's not that difficult.

PC-LN-TAB: in the .gopclntab section



All offsets are relative to their start of their own tab
Except pcOff is relative to textStart

Renaming functions with their original name: steps



- 1 Tell AI to read **CUJO blog** 
- 2 AI finds .gopclntab
- 3 AI writes a script to search for a specific function in PCLNTAB
- 4 Have AI write a Ghidra script to rename all functions
- 5 Enjoy

Step 1 and 2: starting to work with PCLNTAB

Read this <https://cujo.com/blog/reverse-engineering-go-binaries-with-ghidra/> to understand how to find the function.

Step 1

```
% WebFetch https://cujo.com/blog/reverse-engineering-go-binaries-with-ghidra/
```

The article describes the `pclntab` structure: each `functab` entry is `{function_entry_PC, offset_to_function_metadata}`, and the function metadata contains an offset to the function name. So I need to:

1. Find the `functab` in `.gopclntab`
2. Walk entries until the name offset points to `ReadingMessages`

The `.gopclntab` starts at `0x08410ec0`. From the header: `nfunc=0x188C`. The `functab` starts after the `header+offsets`. Let me read the `functab` and find the entry whose name offset resolves to `ReadingMessages`.

Step 2



Step 3: Get the address of ReadingMessages

header+offsets. Let me read the functab area and find the entry whose name offset resolves to `ReadingMessages`.

```
$ python3 -c "
import struct

# From binary: .gopclntab = 0x08410ec0
# Header bytes (from Ghidra inspect):
# F1 FF FF FF 00 00 01 04 8C 18 00 00 A6 02 00 00
# 00 90 04 08 40 00 00 00 E0 BC 04 00 00 FA 04 00
# 00 73 05 00 E0 24 11 00 ...

# magic=0xFFFFFFFF1 (Go1.20), minLC=0, quantum=0, ptrSize=4 (32-bit)
# nfunc = 0x188C = 6284
# After header (16 bytes) come 8 pointer-sized table VA/offset entries

# From the hex dump at offset +0x10:
# 00 90 04 08 -> LE = 0x08049000 -- looks like an absolute VA
# 40 00 00 00 -> 0x00000040      -- relative offset 0x40 from pclntab base
# E0 BC 04 00 -> 0x0004BCE0
# 00 FA 04 00 -> 0x0004FA00
# 00 73 05 00 -> 0x00057300
# E0 24 11 00 -> 0x001124E0

# In Go 1.20 pclntab these 8 entries are all relative offsets from pclntab start:
```

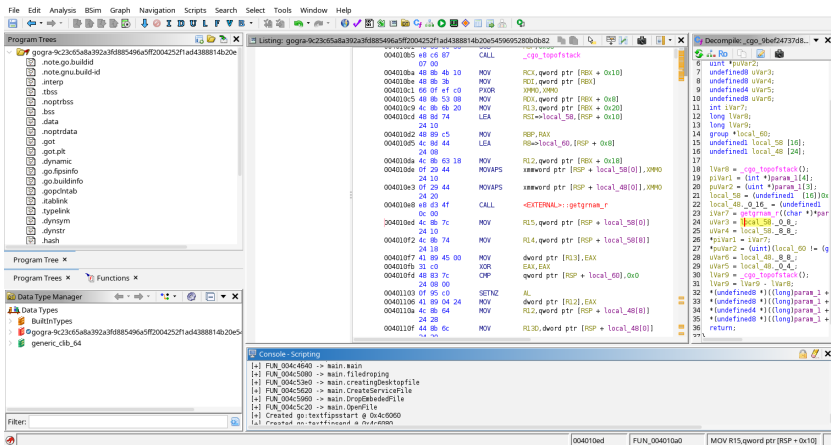
The name resolves perfectly. `services.ReadingMessages` is at `0x08286660`.

```
@ ghidra_rename_function_by_address [function_address=0x08286660, new_name=services_ReadingMessages]
```

```
services.ReadingMessages -> 0x08286660
```



Step 4: Script it for Ghidra



Ghidra in the process of renaming all functions, using a script generated by AI



Step 5: Enjoy recovered function names

```
[+] os/user.Current.func1 -> os/user.Current.func1
[+] type:.eq.os/user.User -> type:.eq.os/user.User
[+] main.main -> main.main
[+] main.filedropping -> main.filedropping
[+] main.creatingDesktopfile -> main.creatingDesktopfile
[+] main.CreateServiceFile -> main.CreateServiceFile
[+] main.DropEmbeddedFile -> main.DropEmbeddedFile
[+] main.OpenFile -> main.OpenFile
```

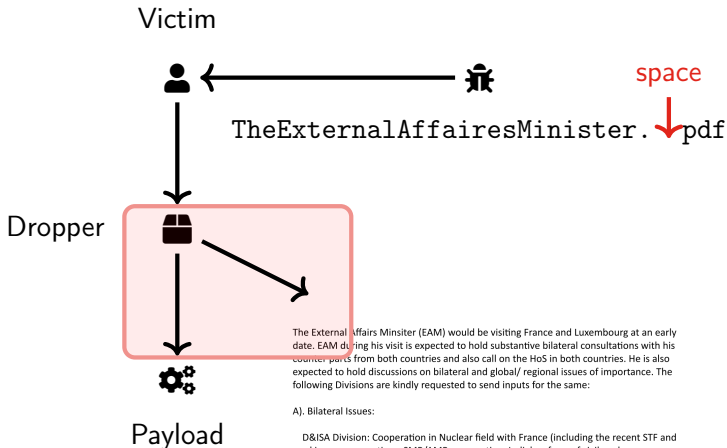


Inside Linux GoGra backdoor

- ① Introduction
- ② Reversing Go binaries: strings, .gopclntab
- ③ **Dropper's extraction mechanism**
- ④ Executing the payload
- ⑤ Persistence
- ⑥ Communication through encrypted e-mails



Details of the Extraction mechanism



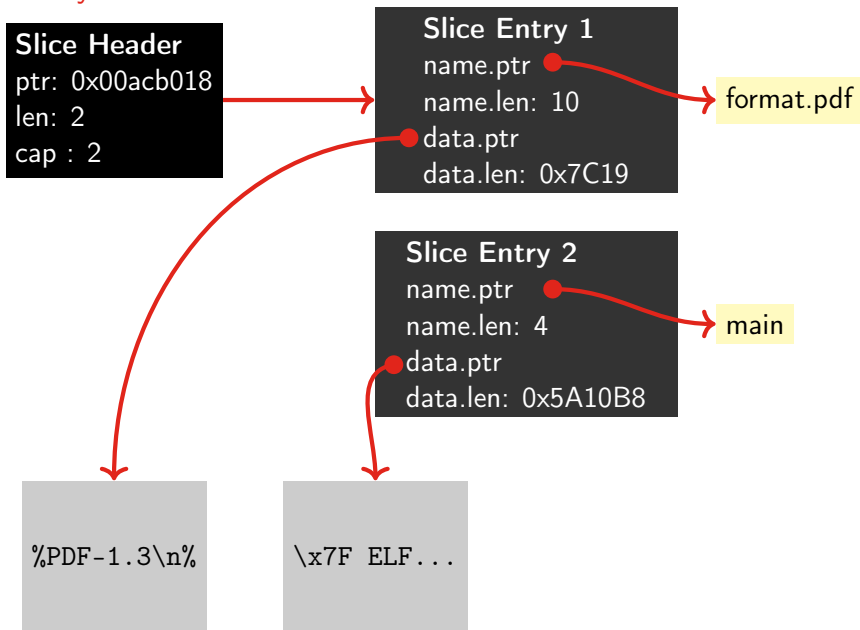
Embedding files in Go

go:embed to include static files and directories

```
import _ `embed`  
  
//go:embed main  
//go:embed format.pdf  
var embeddedFS embed.FS
```

- `//go:embed`  is a **compiler directive comment**.
- It is binded to the *next* line, `embed.FS`.
- It reads `main` and `format.pdf` at compile time.

At binary level



Example of Slice Entry

		PTR_s_format.pdf_00acb018	
00acb018	6d 40 4f	addr	s_format.pdf_004f406d
	00 00 00	←	pointer to name
	00 00		
00acb020	0a	??	0Ah
00acb021	00	??	00h
00acb022	00	??	00h
00acb023	00	??	00h
00acb024	00	??	00h
00acb025	00	??	00h
00acb026	00	??	00h
00acb027	00	??	00h
00acb028	e0	??	E0h
00acb029	de	??	DEh
00acb02a	4f	??	4Fh
00acb02b	00	??	00h
00acb02c	00	??	00h
00acb02d	00	??	00h
00acb02e	00	??	00h
00acb02f	00	??	00h
00acb030	19	??	19h
00acb031	7c	??	7Ch
00acb032	00	??	00h
00acb033	00	??	00h

data size

Screenshot of Ghidra showing the slice entry for format.pdf file.



Extraction Demo

```
ExtractPayload.java> Running...
[*] ExtractPayload: embed.FS at VA 0xb6ee20
[*] embed.FS.files ptr = 0xacb000
[*] File table: data=0xacb018 len=2 cap=2
[*] Entry 0: name='format.pdf' data=0x4fdee0 size=31769
↳ hash=e8114d6a18220998a1546cbbd633ce52
  [*] Read 31769 / 31769 bytes from VA 0x4fdee0
  [+] Written to: ./artifacts/format.pdf
  [+] File type: PDF
[*] Entry 1: name='main' data=0x505b00 size=5902520
↳ hash=e2f8b227e8b1046921acfd15acd06f32
  [*] Read 5902520 / 5902520 bytes from VA 0x505b00
  [+] Written to: ./artifacts/main
  [+] File type: ELF executable

[*] Done.
ExtractPayload.java> Finished!
```



Different versions of Decoy PDF

Filename	Decoy PDF description	Presumed Target
format.pdf	Fake internal circular from Indian Ministry of External Affairs , requesting inputs for a presumed visit of External Affairs Minister to France and Luxembourg.	India
TheExternal-Affaires-Minister.pdf	External Affairs Minister document. Misspelling on <i>Affaires</i>	India
Zomato Pizza.pdf	Fake food delivery lure	India
umrah.pdf	Islamic pilgrimage document	Afghanistan

Inside Linux GoGra backdoor

- ➊ Introduction
- ➋ Reversing Go binaries
- ➌ Dropper's extraction mechanism': embed.FS, extract
- ➍ **Executing the payload**
- ➎ Persistence
- ➏ Communication through encrypted e-mails



How are the files used / executed?

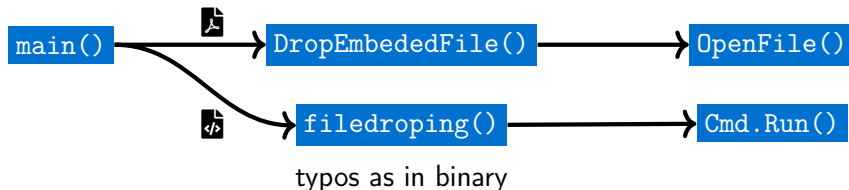


The dropper extracts two files:

- 1 `format.pdf`
- 2 `main`

Now, what?

I asked a LLM to investigate



LLM's analysis

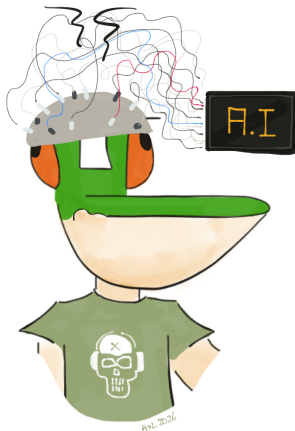
"OpenFile executes the previously dropped payload at <cwd>/<name>.pdf, waits for it to finish, captures and prints its output, then removes the file from disk. The name "OpenFile" is a deliberate misdirection — it does not open a file for reading; it executes it."

Source code generated by AI, based on assembly

```
func OpenFile(name string) {  
    // 1. Reconstruct full path of the dropped payload  
    cwd, _ := os.Getwd()  
    payloadPath := cwd + "/" + name + ".pdf"  
    fmt.Println(payloadPath) // logged  
  
    // 2. Execute the payload and capture output  
    //     os.exec.Command(payloadPath) - no arguments  
    ↪     passed  
    //     A new bytes.Buffer is allocated and wired to  
    ↪     cmd.Stdout  
    cmd := exec.Command(payloadPath)
```

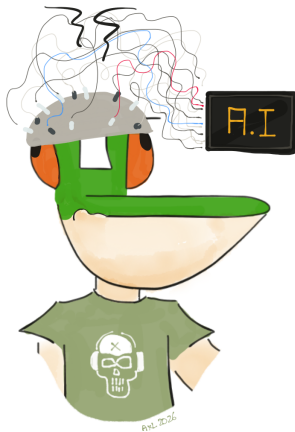
Time to Think

- It wouldn't be the first time we see **deliberate misdirection** in a malware



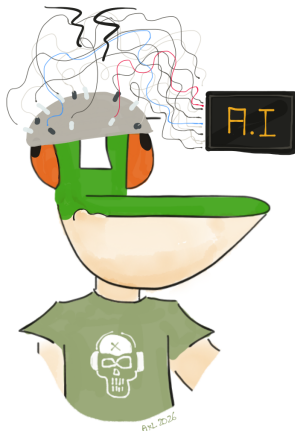
Time to Think

- It wouldn't be the first time we see **deliberate misdirection** in a malware
- It's the goal of a **dropper to drop and execute executables**



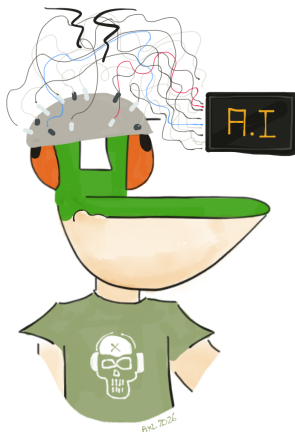
Time to Think

- It wouldn't be the first time we see **deliberate misdirection** in a malware
- It's the goal of a **dropper to drop and execute executables**
- Hey, it's **executing the PDF?!**



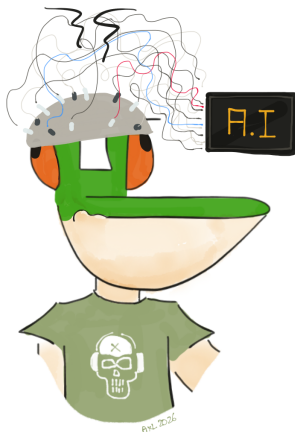
Time to Think

- It wouldn't be the first time we see **deliberate misdirection** in a malware
- It's the goal of a **dropper to drop and execute executables**
- Hey, it's **executing the PDF?!**
- It wouldn't be the first time **extension names are misleading**



Time to Think

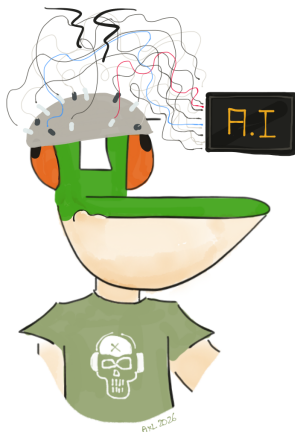
- It wouldn't be the first time we see **deliberate misdirection** in a malware
- It's the goal of a **dropper to drop and execute executables**
- Hey, it's **executing the PDF?!**
- It wouldn't be the first time **extension names are misleading**
- So, we have **two executables**: `main` and `format.pdf`?



Time to Think

- It wouldn't be the first time we see **deliberate misdirection** in a malware
- It's the goal of a **dropper to drop and execute executables**
- Hey, it's **executing the PDF?!**
- It wouldn't be the first time **extension names are misleading**
- So, we have **two executables**: `main` and `format.pdf`?
- But `format.pdf` is really a PDF.

```
format.pdf: PDF document, version  
1.3, 1 pages
```

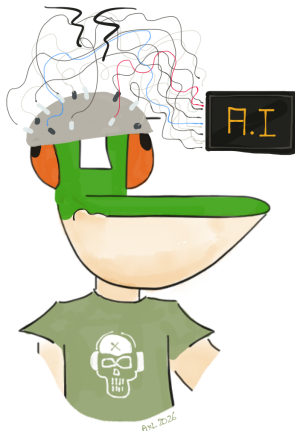


Time to Think

- It wouldn't be the first time we see **deliberate misdirection** in a malware
- It's the goal of a **dropper to drop and execute executables**
- Hey, it's **executing the PDF?!**
- It wouldn't be the first time **extension names are misleading**
- So, we have **two executables**: `main` and `format.pdf`?
- But `format.pdf` is really a PDF.

```
format.pdf: PDF document, version
1.3, 1 pages
```

- You can't execute a PDF?!



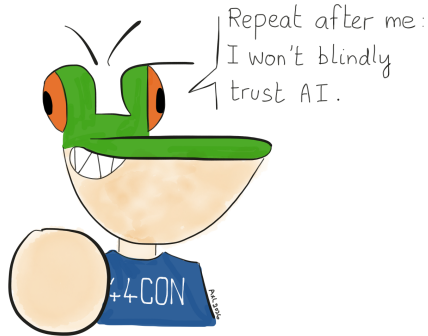
Check it in Ghidra

```
sVar11.len = 0xb;  
sVar11.str = "libreoffice";  
arg.len = 1;  
arg.array = &local_20;  
arg.cap = 1;  
local_20 = sVar9;  
self = os/exec.Command(sVar11,arg);
```

- sVar11 is libreoffice
- arg is cwd/format.pdf
- Executes Command("libreoffice","cwd/format.pdf")

Opens the decoy PDF with LibreOffice





The LLM **assumed** a payload would be executed,
because it was likely.

Cross examine what any LLM says. Ask the same questions to different LLMs. Ask the LLM to prove what it claims.

Claim no.2 from AI



*"This is a smart fileless mechanism. The PDF and the ELF payload are **in memory**, never written to the file system."*

embed.FS.ReadFile reads in memory.

004c599d	0f 1f 00	NOP	dword ptr [RAX]
004c59a0	e8 bb 25 fd ff	CALL	embed::embed.FS.ReadFile

Why does it *remove the file* then?

004c5f24	41 bb 05 00 00 00	MOV	R11D,0x5
004c5f2a	e8 11 5f f9 ff	CALL	runtime::runtime.concatstring4
004c5f2f	e8 2c 96 fe ff	CALL	os::os.Remove

Look at the assembly: `filedropping()` and `DropEmbeddedFile()`

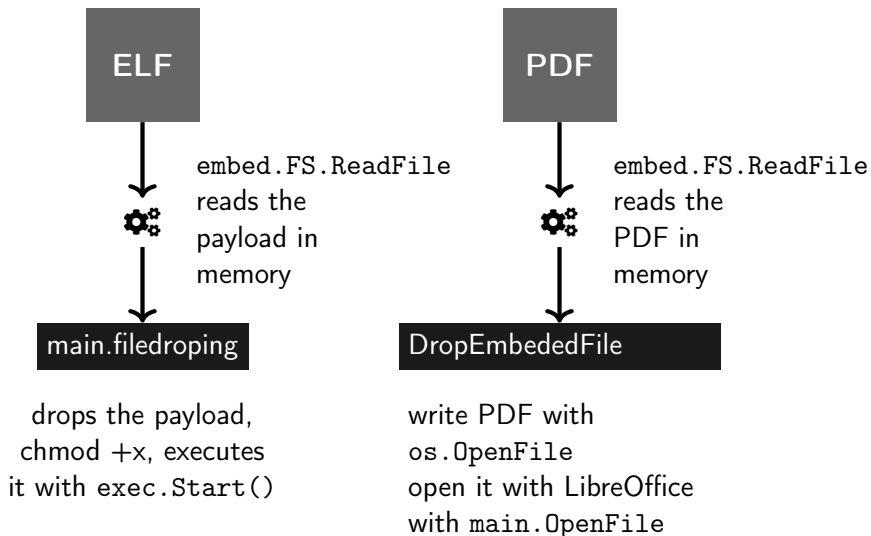


Both files are opened with `O_WRONLY` (`0x01`) and `O_CREATE` (`0x40`).
This creates the file!

```
004c5abf  99 41 00      MOV     ECX,0x41
004c5ac4  bf ff 01      MOV     EDI,0x1ff
004c5ac9  e8 f2 8d      CALL    os::os.OpenFile
fe ff
```

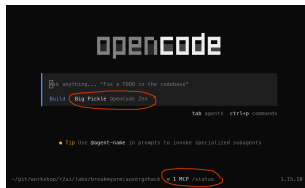
AI is wrong. This is not a fileless mechanism at all.

How it really works



AI is not all Good or all Bad

When AI claims something, **check it**
with your own eyes in assembly



When assembly is difficult to understand, **ask AI** to clarify

Inside Linux GoGra backdoor

- ➊ Introduction
- ➋ Reversing Go binaries
- ➌ Dropper's extraction mechanism
- ➍ Executing the payload
- ➎ **Persistence**
- ➏ Communication through encrypted e-mails



How the malware achieves persistence



User Systemd service

- Runs as soon as the user logs in
- Systemd is widely used, but not everywhere: Alpine Linux traditionally uses *OpenRC*.
- `~/.config/systemd/-user/userservice`



XDG autostart

- Mechanism to start application in a Window Manager.
- Only works for GUI logons
- `~/.config/autostart-abc.desktop`

Systemd unit file

```
a1.len = iStack0000000000000020;  
a1.str = pcStack0000000000000018;  
a0_00.len = 0x31;  
a0_00.str = "[Unit]\nDescription=Foo\n[Service]\nExecStart=/home/";  
a2.len = 0x66;  
a2.str =  
"/.config/systemd/user/userservice\nRestart=always\nRestart=on-failure\n[Install]\nWantedBy=  
t.target "  
;  
sVar3 = runtime::runtime.concatstring3((runtime.tmpBuf *)0x0,a0_00,a1,a2);  
[Var6 = runtime::runtime.stringtoslicebyte(&local_c8,sVar3);  
mVar7 = os::os.(*File).Write(local_a0,([]uint8)[Var6];
```

```
[Unit]  
Description=Foo <----- not a good idea  
[Service]  
ExecStart=/home/<user>/.config/systemd/user/userservice  
Restart=always  
Restart=on-failure  
[Install]  
WantedBy=default.target
```



XDG autostart file

```
[Desktop Entry]
Type=Application
Name=conky
Exec=/home/<user>/.config/systemd/user/userservice
```

Poses as *Conky* , a legitimate desktop system monitor

"main" is copied to ~/.config/systemd/user/userservice

004c51ae(R)

main::main.fileddropping

XREF[4]:

All comments added
by AI, through MCP

new Dropper.go:85 (26)

```
004c5080 4c 8d 64 LEA R12=>local_30,[RSP + -0x30]
24 d0
004c5085 4d 3b 66 10 CMP R12,qword ptr [R14 + 0x10]==>CURRENT_G.stackguard0
004c5089 0f 86 ff JBE LAB_004c538e
02 00 00
004c508f 55 PUSH RBP
004c5090 48 89 e5 MOV RBP,RSP
004c5093 48 81 ec SUB RSP,0xa8
a8 00 00 00
004c509a 48 89 b4 MOV qword ptr [RSP + Stack[0x28]],RSI
24 d8 00
00 00
004c50a2 48 89 bc MOV qword ptr [RSP + Stack[0x20]],RDI
24 d0 00
00 00
004c50aa 48 89 9c MOV qword ptr [RSP + Stack[0x10]],RBX
24 c0 00
00 00
004c50b2 48 89 84 MOV qword ptr [RSP + Stack[0x8]],RAX
24 b8 00
00 00
004c50ba 48 89 8c MOV qword ptr [RSP + Stack[0x18]],RCX
24 c8 00
```

stack guard: frame top = RSP-0x30

cmp with g.stackguard0 (R14 = g)
stack overflow? jump to morestack...

push RBP
RBP = RSP
allocate 0xa8-byte frame

save arg: payload ptr (embedded ...

save arg: len('userservice') = 11

save arg: configdir len

save arg: configdir ptr ('/home/...

save arg: filename ptr 'userserv...

All comments are now in Ghidra (164 line comments + plate comments on main.main, main.fileddropping, main.CreateServiceFile).



Dropper's main - presumed code from AI

```
func main() {
    // Drop the embedded payload binary to disk and
    // → open/execute it
    DropEmbeddedFile()

    // Get current user info to build home-directory paths
    currentUser, err := user.Current()
    if err != nil {
        fmt.Println(err)
        return
    }
    username := currentUser.Username
    homeDir := currentUser.HomeDir
    fmt.Println(username)
    configDir := homeDir + "/.config"

    // Check if ~/.config/systemd/user/ exists
    systemdUserDir := configDir + "/systemd/user"
    _, statErr := os.Stat(systemdUserDir)
    ...
}
```

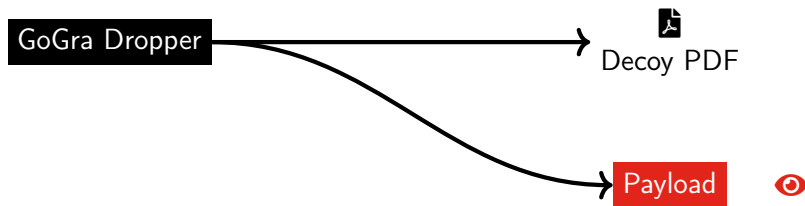


Inside Linux GoGra backdoor

- 1 Introduction
- 2 Reversing Go binaries
- 3 Dropper's extraction mechanism
- 4 Executing the payload
- 5 Persistence: systemctl and XDG
- 6 **Communication through encrypted e-mails**



Let's look into the payload



- We reverse the **payload** now.
- **ELF i386** binary, **stripped**.

Custom Go package

The payload implements a custom Go package named **OUTLOOKCLIENT**:

```
OUTLOOKCLIENT/services.GetCompleteAuth  
OUTLOOKCLIENT/services.GetUser  
OUTLOOKCLIENT/services.CreateFolder  
OUTLOOKCLIENT/services.GetFolderId  
OUTLOOKCLIENT/services.EncryptString  
OUTLOOKCLIENT/services.addPKCS7Padding  
OUTLOOKCLIENT/services.DecryptString  
OUTLOOKCLIENT/services.ExcuteCommand  
...
```



Microsoft Graph REST API

Malware

Microsoft

Hard-coded
tenant Id

`https://login.microsoftonline.com/-
TENANT_ID/oauth2/token`

`GetCompleteAuth()`

OAuth2 token

`GetUser()`

`https://graph.microsoft.com/beta/me`

userId, folderId

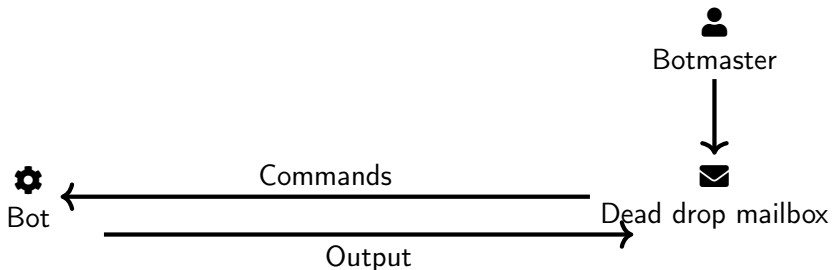
`https://graph.microsoft.com/beta/users/-
USER_ID/mailFolders/FOLDER_ID/messages`

`ReadingMessages()`

message



Communication with C2



This is smart

- 1 Communication occurs through a **legitimate protocol**.
- 2 The **botmaster can change over time**. Bots don't know who is their master.
- 3 OAuth2 token not as easy to trace as an email address.

AI claim no.3 : Emails are encrypted with AES

C2 commands via e-mails ✉

Likely AES-encrypted in the body

```
specific mail folder for new messages. Each incoming email is a C2
command (likely AES-encrypted in the body); the implant executes it
via `/bin/bash` and replies with output as a new email or places it
in another folder. Commands are deleted after reading (
`DeleteingMessage`).
```

Do we have proof? No

```
To directly answer your question: no, we don't have proof that the body is
encrypted – we have proof that AES-256-CBC capability exists in the binary
and is named for C2 use, but static analysis never found a caller of
services_DecryptString, so the call chain from "receive email → decrypt
body" was not traced. The "no callers" finding is the gap. The update
reflects this accurately rather than overclaiming.
```

AI is having a hard time proving emails are encrypted

- 1 LLM clearly identified the AES functions, but **is unable to trace them back to communication with Graph API**
`crypto/aes.NewCipher,`
`crypto/cipher.NewCBCDecrypter...`
- 2 We have the addresses of 2 interesting functions:

Address	Malware function name
0x08286660	OUTLOOKCLIENT/services.ReadingMessages
0x082870e0	OUTLOOKCLIENT/services.SendingMessages

- 3 But AES is not used in ReadingMessages or SendingMessages

Pivoting: providing human intuition to AI

ReadingMessages returns a message body.

💡 Perhaps decryption occurs after.

```
OC | Email-based C2 with AES encryption 134x32

1. Content field comment: "AES-256-CBC encrypted command" → "raw body string;
   presumed AES-256-CBC encrypted (not traced)"
2. End comment: removed the assertion that decryption happens in the caller →
   replaced with the factual observation that no decryption call was seen in this
   function and the caller chain was not traced.

■ Build · Claude Sonnet 4.6 · 35.0s

Where does the implant process the content of the email?

Good question. The call chain we need to trace is: ReadingMessages returns msg.Body.
Content → something receives it → processes/decrypts/executes it. Let me find that
caller.

o ghidra_get_function_callers [address=0x08286660]
o ghidra_get_function_xrefs [address=0x08286660]

■ Build · Claude Sonnet 4.6

⚠ Permission required
  o Call tool ghidra_get_function_xrefs

Tool: ghidra_get_function_xrefs

[ Allow once ] Allow always Reject ctrl+f fullscreen ↵ select enter confirm

Email-based C2 with AES encryption

Context
82,952 tokens
8% used
$6.38 spent

MCP
• ghidra Connected
• r2mcp Connected

LSP
LSPs are disabled

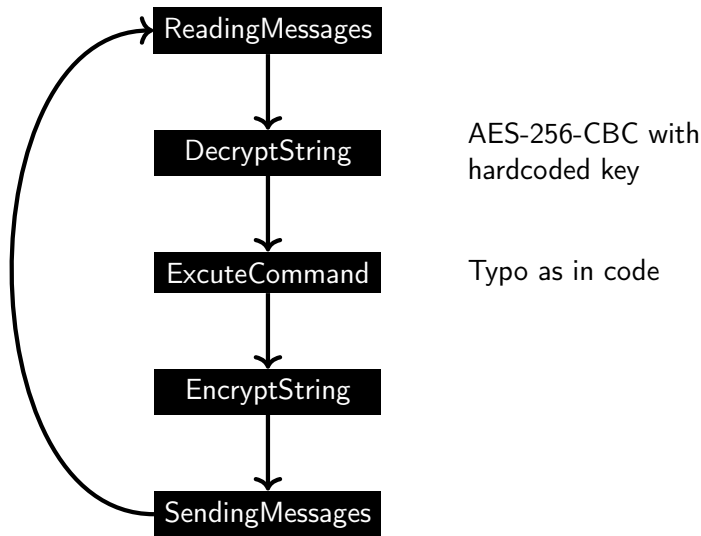
▼ Todo
[✓] Update details.md: add resolved
    function entry addresses for all
    service functions
[✓] Update report.md: add crypto
    section with confirmed AES-256-
    CBC details and function
    addresses
[ ] Update details.md: add
    decompiled/ directory structure
    note

~/samples
• OpenCode 1.15.6
```

This unlocks the situation!



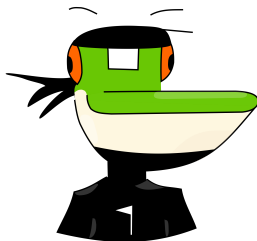
Main loop: encrypting/decrypting C2 emails



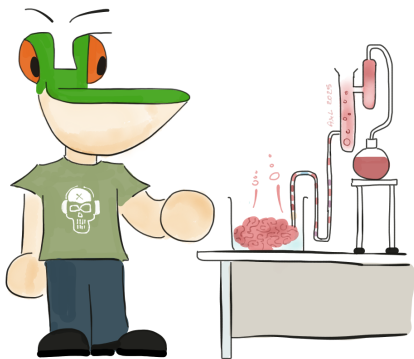
Conclusion



Good news



- Cybercriminals use **Go, Rust, Dart** because they are difficult to reverse
- Decompilers have difficulties processing those languages: specific ABI, specific registers etc
- **Reversing assisted by AI** solves this problem easily



🔑 The key is **not** to let AI drive you
but to **drive** AI

Open your disassembler **first**, or you're doing it wrong.

Wanted to ask this? 1/2



- Can I use a **different disassembler than Ghidra?**

Yes, of course e.g. Radare2 MCP exists too!

- Can I use a **different coding agent than OpenCode?**

VS Code or Claude Code work fine too etc. Configuration differs.

Wanted to ask this? 2/2



- Create a **malware analysis orchestrator agent**?
 - ▶ I have one, but it's good for overview only.
 - ▶ Orchestrator agents will improve. Expect this to work in the future.
- Use a **malware analysis skill**?
 - ▶ I have one, I've used it for this malware. It helps.
 - ▶ But it also increases context size

Cost



- Agents, skills and MCP are **greedy for tokens**
- With Claude, such an analysis costs ~7 pounds. *But you can get the same result with good local models or cheaper ones.* It will probably be slower to reach the same quality. Choose the compromise that suits you.
- Limit token usage: context compression, agents with telegraphic style, disable MCPs you don't use

Thanks for your attention



Workshop at 2pm

- Cryptax on Mastodon, BlueSky, Discord
- aapvrille@fortinet.com
- <https://fortinet.com/blog/threat-research> 🔗

